706 Project 2 Report

Akoh751 vcol141 ckim553 eoxe316

Contents

1. Fire detection and extinguishing system (Vlad) 3
- **Phototransistor Measurements** 3
- **Fire Detection** 3
- **Following Fire** 4
- **Fire Extinguishing** 4

2. Robot behaviour logic (Andrew) 5
- **Implementation** 5
- **Results** 6

3. Obstacle avoidance system (Elise) 7
- **Physical configuration** 7
- **Dealing with sensors readings** 7
- **Avoidance logic** 7
- **Strafing vs Turning** 8
- **Update of Avoidance Logic** 8

4.Hardware and Integration (Christoph) 9
- **Sensor Placement for Obstacle Avoidance** 9
- **Sensor Placement for Fire Detection** 9
- **Hardware and Software Architecture** 10

5. Project Management 11
- **Problem definition** 11
- **Problem decomposition** 11
- **Task Allocation** 11
- **Agile Product Development** 12
- **External Software Management** 12

1. Fire detection and extinguishing system (Vlad)

Phototransistor Measurements

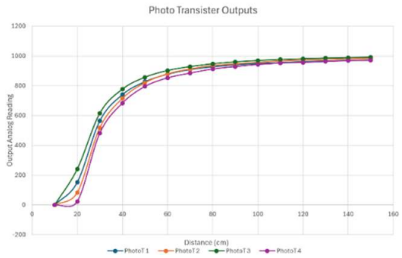

Figure 1 - Raw Phototransistor Readings

We found during testing that the phototransistors did not provide identical readings at fixed distances as shown in figure 1, to make calculations easier we found that the difference between the photo transistor was on average by a fixed factor, so in the code when reading, we used phototransistor one as the baseline, and found the corresponding factor to align them all which provided to be adequate for our need.

Fire Detection

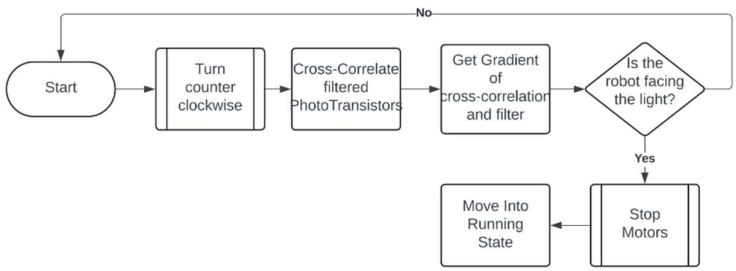

Figure 2 - Fire Finding Logic

Figure 2 shows the logic behind how the robot finds the light. The logic is simple, using the backwards difference of the corss-correlation to detect when a fire has been reached, using the jump in gradient when the robot faces the light. Due to the way the phototransisters are mounted, as shown in section 4, we can assume that each phototransister effectively reads the same values during the sweep but delayed by a factor.

To demonstrate the effects of cross-correlation and backwards difference, figure 3 shows the three stages of the code using pseudo phototransistor input, then filtered cross-correlation, and then backwards difference. We can see that when the phototransistor stops decreasing after 20 seconds, there is a jump in the backwards difference, which can then be used to determine when the robot is facing the light.

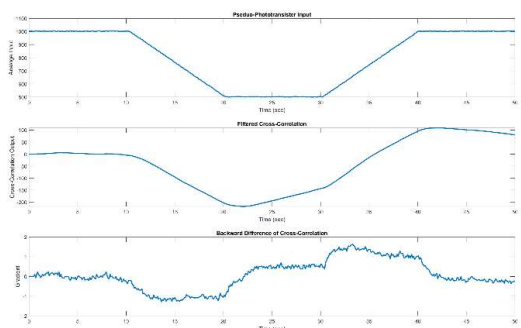

Figure 3 - Pseudo Light detection

We found that the result of this method was robust and was able to work at different distances from the light and didn't have any major issues with glare or reflection. The robot was prone to stopping slightly past the light likely due to delay caused by the filtering but would always stop within an acceptable rand for the Fire following function to work and detect the light.

Following Fire

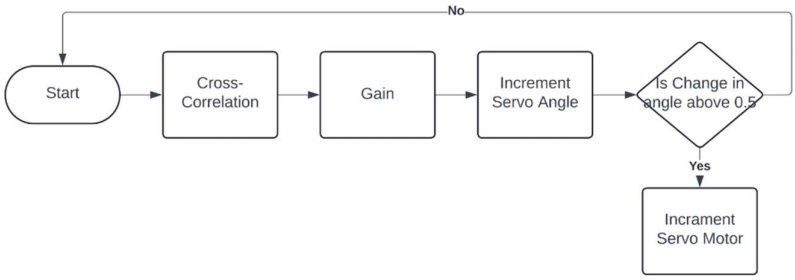

Figure 4 – Fire Following Logic

Similar to the fire detection logic, the fire following logic uses cross-correlation. The following logic uses cross-correlation but tries to keep the phototransistor values balanced, as any imbalance across the phototransistors leads the cross-correlation rising above zero or lowering below zero.

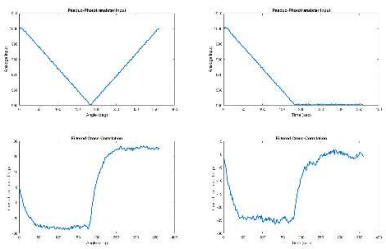

Figure 5 - Pseudo Light Following

The left-hand side of figure 5 shows how the cross-correlation reacts if the robot spins 360°, with the output being negative when the light is getting closer, and positive when moving away, and the right-hand side showing the response if the robot stays facing the light once turned towards it. From this logic we can use the cross-correlation as the error for a basic KP controller where the controller wants to try keep the cross-correlation zero.

The go to light logic was a basic KP controller where the reference was 90, and the error was the current position of the servo, so that the robot would try to always face the fire while driving forwards. This approach worked well, and with extra functionality to reduce forward motion when the error was too larger it made sure that the robot took priority of facing the fire over the speed of travel.

This approach worked well, able to find the light and approach while also working with the anti-collision behaviour. We found that there were circumstances where the light would be lost if the robot had to turn 180° to get out of a tight space with obstacle avoidance but would eventually find the light again. We also found that during the run if the fire was too close to the corner of the table, the cross-correlation would start to act sporadically due to the glare off the somewhat reflective wall surface leading to the fan not being faced correctly, we found that by putting a mat surface on the wall this issue was mitigated, and tighter constraints could further mitigate this issue, but due to the issue not occurring during testing this was not able to be implemented.

Fire Extinguishing

To detect when the fire had been reached, the code simply checked the average reading between the two forward phototransistors and if they were under a threshold (indicating the fire was close to the robot) the avoid functionality would be disabled in order so that the robot didn't try and avoid the fire, and the robot would stop once the sonar or two front IR's were in range, then the robot would stop and the fan would turn on, once the fire had been put out the robot would restart in the fire detection sequence to find the next fire.

2. Robot behaviour logic (Andrew)

Our previous robot had issues with code structure, cleanliness, and ease of implementing new functionality. Because of this, we chose to implement a behaviour-based control framework to clean up our robot behaviour logic and allow us to easily add new functionality into our robot code.

Our behaviour-based control simulates a 'think the way you act' behaviour inspired by nature and animal brains. This structure allows the robot to be able to react to new inputs and act accordingly in real-time. Because our robot must be autonomous and complete its task in an unknown environment in minimal time, speed and simplicity is what we prioritized.

By nature of the structure in which behaviour-based control is implemented, this framework allows us to resolve conflicting sensor data and behaviours by processing all behaviours after sensor inputs are taken. Instead of immediately doing the action associated with a condition, the framework instead raises a flag telling the robot what its next action must be. This also allows us to prioritize certain behaviours over other based on the order in which we place our behaviour conditions. Furthermore, because we implement our behaviours/conditions incrementally, we can add on new features and behaviours quickly and test them individually.

It can be noted that though behavioural control typically runs its behaviours in parallel with each other, our microcontroller is single core and thus is incapable of doing this. However, this proves to not be an issue within the scope of what our robot needs to achieve as its computations are simple and its speed allows for enough time to run through all sensors.

Implementation

Below is a snippet of the main 'RUNNING' loop for behavioural control.

```
STATE running() {
  update_transistors();
  read_IR_sensors();
  filter_IR_reading();


  move_forward();
  avoid();
  put_out_fire();
  all_fires_extinguished();
  arbitrate();

  return RUNNING;
}
```

Figure 6 All behavioural control functions in the main loop

The five functions towards the bottom make up the structure of our behavioural control where the functions above 'arbitrate' are simply condition checks that will set behaviour flags and their respective commands for the robot to follow once it processes these flags in the arbitrate function.

In addition to this, instead of reading each sensor's measurement variable individually where each condition will need to have a threshold, we decided to use one 5-bit binary variable containing all five of our sensors. The enum and variable definition can be shown below. To fill the variable, we created a function to continuously read all our sensors and compare it to a specific threshold which would flip the sensor's bit depending on the reading.

```
/********IR BINARY***********/
enum IR_BINARY{
  LR3,
  LR1,
  MR2,
  MR1,
  SONAR
};

//IR Binary Variable
//Convention: LR3,LR1,MR2,MR1,Sonar
unsigned int IR_bin = 0b00000;
```

Figure 7 Binary Variable for all sensors

This allowed us to more easily define when the robot had sensed an obstacle in front of it by reading a 1 or 0 from our desired bit when writing our conditional functions. A snippet for one of these functions can be shown below.

```
void avoid(){
  float middle_avg = (photo2_avg+photo3_avg)/2;
  if((!check_bits(LR1) && !check_bits(LR3) && !check_bits(SONAR))){
    avoid_flag = false;
    avoid_command = NULL;
  }
  else if(check_bits(MR1) && check_bits(MR2)){
    avoid_flag = true;
    avoid_command = TURN_RIGHT;
  }else if((check_bits(MR1) || (!check_bits(LR3) && check_bits(LR1)))){
    avoid_flag = true;
    avoid_command = TURN_RIGHT;
  }else if (avoid_command != TURN_RIGHT){
    avoid_flag = true;
    avoid_command = TURN_LEFT;
  }
}
```

Figure 8 Simplified version of our 'Avoid' conditional function to set flags and movement commands

Here we can see each conditional check that the sensors must go through when the avoid function is run. If a condition is met the avoid flag is raised to true and its corresponding movement command is also set. In other words, if the sensors detect an object, the robot is signalled to avoid and will follow a movement command to avoid the object.

```
void arbitrate(){
    if(forward_flag == true){
        motor_input = forward_command;
    }
    if(avoid_flag == true){
        motor_input = avoid_command;
    }
    if(fire_flag == true){
        motor_input = fan_command;
    }
    if(stop_flag == true){
        motor_input = stop_command;
    }
    robot_move();
}
```

Figure 9 Code snippet of 'Arbitrate'

The 'arbitrate' function, as mentioned earlier, will process these behaviour flags by checking which one is raised and set the robot's next movement command to be the one that was set by the conditional functions. The code for 'arbitrate' is shown to the left.

Finally, the 'robot_move' function is run which simply runs the 'motor_input' variable.

Below is the flow diagram for behavioural control including all sensors, actuators, and behaviours our robot follows. Included are coloured lines for each behaviour, showing the flow and priority of each.

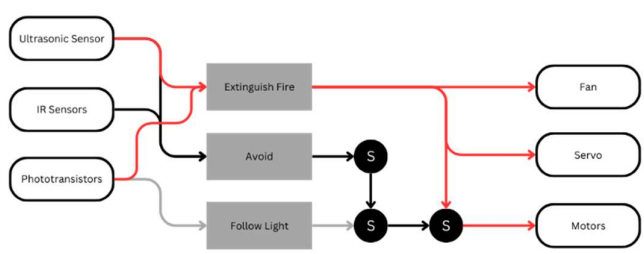

Figure 10 10 System Architecture from viewpoint of behavioural control

Results

The results of using behavioural control allowed us to better structure our code leading to an easier time fixing bugs and work in a more organised manner.

Because functions and behaviours can be added on incrementally in the behavioural control framework, we were able to develop and test each function individually, thus making sure all functions worked before combining them together.

Our robot was able to perform much more predictably compared to the previous project, and, though there is no notable change in performance, allowed better workflow within our group.

3. Obstacle avoidance system (Elise)

To fulfill the project's objectives, a means to detect obstacles and react accordingly to avoid collision was required. For this system, we needed to determine the physical configuration of the sensors, gather usable readings and write logic for the robot to deal with these.

Physical configuration

To detect obstacles, an ultrasonic sensor and four IR sensors (two long-range (LR) and two mid-range (MR)) were used. For further reasoning behind implementation, refer to Section 4.

Dealing with sensors readings

To quantify the IR readings to a distance value, the average readings for each sensor were taken at different distances and an equation was fitted to these results. As we were only concerned with object detection at shorter distances, readings were taken up to 25cm away for MRs and up to 40cm away for LRs.

To make the readings from the sensors more reliable, we added a Kalman filter for the IRs and the ultrasonic sensor. However, we found that this reduced the reactivity of the system, meaning objects were often detected too late, often resulting in a collision. When the filtering was removed, we found that the raw readings were adequate to deal with the task.

Avoidance logic

Using the raw reading, a Boolean value was assigned to each sensor, where 1 indicated an obstacle had been detected, and 0 if not. If the raw reading was below a given threshold, its relevant Boolean value would be equal to true. For the sonar, this value was 15cm, LRs used 13cm and MRs used 10cm.

To decide what the robot should do for a set of Boolean variables, we made a K-map of each scenario and the desired response of the robot.

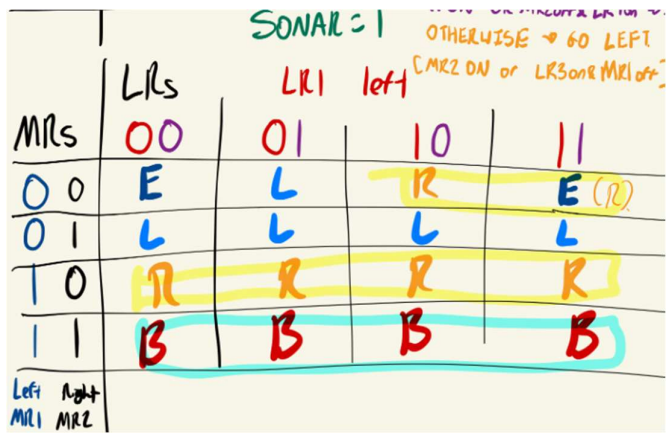

Figure 11 Part of the K-map to determine obstacle avoidance

From this, the avoidance logic was as follows:

- If nothing is detected ahead, keep going straight. (After this it can be assumed a strafing action is needed)
- If both MRs detect a wall, there is no room to strafe. Move backwards.
- If the left MR sees a wall, we cannot move left. Strafe to the right
- The left LR sees an obstacle and the right MR does not. There is space to move to the right and this is the quickest way to avoid the obstacle. Strafe to the right
- Otherwise, strafe to the left.

Strafing vs Turning

When testing our initial avoidance logic, we found that although the robot acted correctly, it would sometimes get stuck. This would happen when the robot got too close to a wall and tried to avoid it. The robot would strafe across the wall as there was room to move to the sides but could never get itself out as it didn't know to backtrack and escape. To deal with this, we changed the logic to turn the robot instead of strafing. This allowed the robot to escape from this situation, as it could turn until it found a way out.

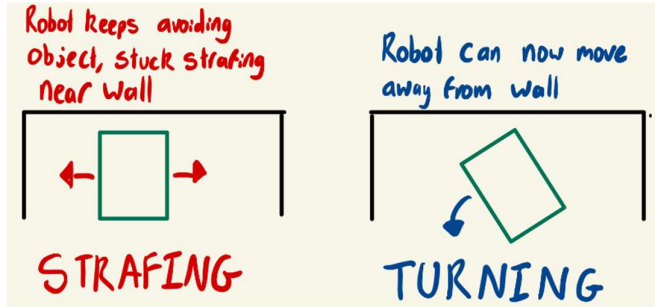

Figure 12 Strafing getting stuck solution illustrated

When using this approach, we found that we didn't need to be as concerned with objects to the side of the robot, as these were picked up by the LRs. This simplified our avoidance logic to only deal with the front three sensors.

Update of Avoidance Logic

As a result of further testing, we made a few additional changes to the avoidance logic:

- Sometimes the robot would keep switching between turning left and right. This generally occurred when the robot ended up in a corner. To fix this, the robot was unable to send a turn right command directly after a turn left command, and vice versa.
- When integrating this system with the following light function, sometimes the robot would lose the light when avoiding an obstacle. To resolve this, if the servo was already at a significant angle, the robot would choose to turn in a manner to help realign the servo.
- When the detected light signal is strong enough, we know the oncoming obstacle was a fire. A condition to the obstacle avoidance was added to not react if this was the case.

```
void avoid(){
  float middle_avg = (photo2_avg+photo3_avg)/2;
  if((!check_bits(LR1) && !check_bits(LR3) && !check_bits(SONAR)) || middle_avg < 600){
    BluetoothSerial.println("NO AVOID REQUIRED");
    avoid_flag = false;
    avoid_command = NULL;
  } else if(((!check_bits(LR3) && check_bits(LR1)) || (currentAngle < 70)) && avoid_command != TURN_LEFT){
    avoid_flag = true;
    avoid_command = TURN_RIGHT;
  } else if (avoid_command != TURN_RIGHT){
    avoid_flag = true;
    avoid_command = TURN_LEFT;
```

Figure 13 Final Code for Obstacle Avoidance

4.Hardware and Integration (Christoph)

Sensor Placement for Obstacle Avoidance

In order to ensure that all software is implemented flawlessly, the hardware was integrated to be functionally and structurally sound. Since this scenario required the robot to navigate tight spaces within less than half of its respective width and length (7cm, 12cm), sensors were mounted where contact was most likely, coinciding with the most common direction of travel.

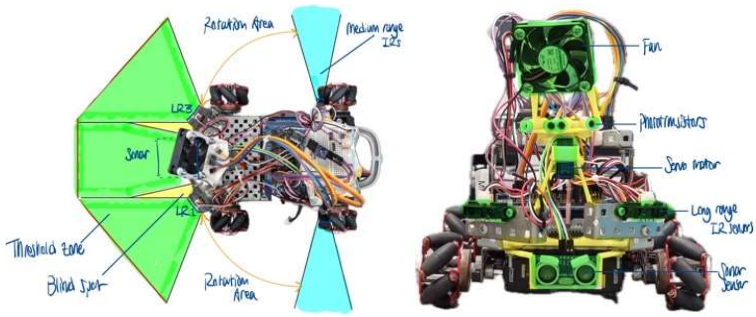

Figure 14.1 and 14.2 Sensor placement

Since our obstacle turned instead of strafing (Section 3) it was clear that the most common moving part was the front end. Thus, the LR sensors were placed on the front end (Figure 14.1), and configurated from a 80cm down to 40cm maximum (Figure 16) for more accurate readings where the robot would react to obstacles without jitter or delay. The LR sensors could not cover the middle of the robot path, hence an ultrasonic sensor was placed at this position (Figure 14.2), especially as it produced the most accurate sensor readings and did not require manual calibration through software. This was crucial in stopping the fan from turning on prematurely, at thresholds larger than 15cm (Figure 15). However, we found obstacles on the edges of the robot's path were often missed due to blind spots, so these LR sensors were turned horizontally (Figure 14.1, 14.2), widening the field of visibility. Due to turning, our robot experienced minimal issue with collisions on its side, so MR sensors were placed on the sides of the robot, vertically, and only configured to 5-25cm and a threshold of 10cm (Figure 16), to avoid oncoming obstacles only when turning could not do so, and prevent unnecessary jittering during the turning obstacle avoidance.

```
void conv_binary(IR_BINARY binary_type, double reading){
  double threshold = 10; //Default to MR threshold
  if(binary_type == SONAR){ //If no MR change to other relevaant threshold
    threshold = 15;
  } else if (binary_type == LR3){
    threshold = 12;
  } else if(binary_type == LR1){
    threshold = 12;
  } else if(binary_type == MR2){
    threshold = 10;
  } if(reading <= threshold){
    IR_bin |= (1 << binary_type); //flip its respective bit
  } else{
    IR_bin &= ~(1 << binary_type); //else if not under
  }
}
```

Figure 15 Sensor thresholds

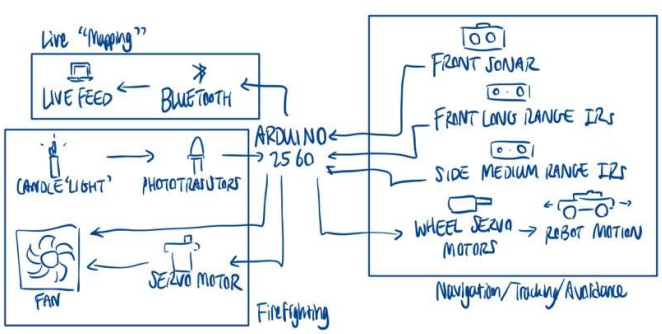

Figure 16 System Architecture for Hardware

Sensor Placement for Fire Detection

Four phototransistors were connected as an analog input, where two were placed as in Section 1 to mitigate false readings, especially in the most important direction (Figure 14.2). The other two phototransistors were connected as inputs, left and right, to increase the field of view during fire detection (Section 1). The direction of the fire was unknown, and the movement of the robot was limited due to time and space, hence the phototransistors were placed on a servo motor on a lightweight 3D printed mount. This allowed not only our obstacle avoidance software to be run simultaneously to the fire detection to reduce time, but also removed gyroscope use, improving sensor reliability and Bluetooth output. Initially, 10kOhm resistors only covered up

to a measured 50cm. A 10MOhm resistor was used instead for each phototransistor because it created a larger voltage difference against light, increasing the sensitivity to the entire field length.

The fan was also connected to the same servo mount so that its orientation did not depend on the movement of the entire robot, but rather the phototransistor readings only. A voltage regulator was used to prevent the fan from breaking.

Hardware and Software Architecture

Pin No.	Feed	Devices	Signal Type	Function	Symbol
10	Output	Bluetooth	Digital	Mapping	BLUETOOTH_RX
11	Output	Bluetooth	Digital	Mapping	BLUETOOTH_TX
1	Output	Bluetooth	Digital	Data transmission	OUTPUTBLUETOOTHMONITOR
A5	Input	Long IR sensor 1	Analog	Range 10~40cm	LR1pin
A6	Input	Long IR sensor 3	Analog	Range 10~40cm	LR3pin
A4	Input	Medium IR sensor 1	Analog	Range 5~25cm	MR1pin
A7	Input	Medium IR sensor 1	Analog	Range 5~25cm	MR2pin
A12	Input	Phototransistor 1	Analog	Field length light detection	photo_pin1
A13	Input	Phototransistor 2	Analog	Field length light detection	photo_pin2
A14	Input	Phototransistor 3	Analog	Field length light detection	photo_pin3
A15	Input	Phototransistor 4	Analog	Field length light detection	photo_pin4
8	Output	Servo motor	Digital	Phototransistor and fan positioning (0~360°)	Servo
21	Output	Fan	Digital	Fan on and off	fan_pin
48	Input	Ultrasonic sensor	Digital	Triggering US sound pulses	TRIG_PIN
49	Output	Ultrasonic sensor	Digital	High when equal US signal received	ECHO_PIN
47	Output	Wheel servo motor	Digital	Positioning (0~360°)	left_front
48	Output	Wheel servo motor	Digital	Positioning (0~360°)	left_rear
50	Output	Wheel servo motor	Digital	Positioning (0~360°)	right_rear
51	Output	Wheel servo motor	Digital	Positioning (0~360°)	right_front

Figure 17 Pinout table

Aside from sensor applications, these inputs (Figure 17) were processed with respective outputs to our software (Figure 18). Through the Arduino Mega microcontroller, our FSM (Section 3) allowed robot intelligence in our outputs (Figure 17).

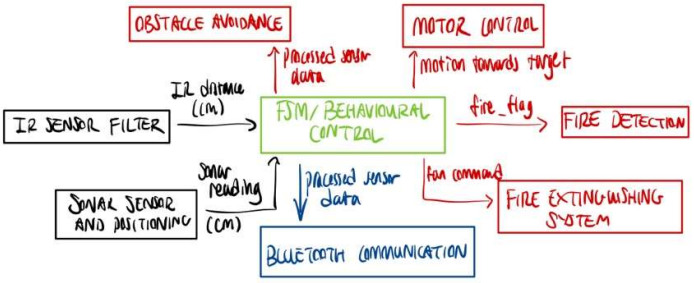

Figure 18 System architecture in terms of function

During testing however, as the servo only had a voltage rating up to 10V upon a 0.5cm diameter shaft, the phototransistors were extremely vulnerable to the vibrations caused by the movement caused by the 12V 4-wheel drive robot's torque. This displaced all sensors, causing our input to be corrupted on our Bluetooth output (Figure 18). This caused outputs in all software (Section 1, 2, and 3) to actuate under false readings. Hence secure housings were designed to the exact diameter and width of each phototransistor and sonar sensor respectively. As well as the IRs and the fan being bolted on. Ultimately, ensuring structural rigidity in our inputs caused our software architecture to perform as functionally designed.

5. Project Management

In this section, we cover our various project management techniques learned from ENGGEN303 which we implemented for this project. To facilitate the progress of our project, we used an Agile product development methodology combined with various communication and management software.

Problem definition

First, we defined our key requirements for the project by writing it in the form of a brief problem statement. This being:

"Utilising our current robotic system which has limited sensors, how can we navigate and extinguish two light sources whilst avoiding all obstacles in its path"

This allows us to set a clear objective in our project for us to achieve whilst breaking down the problem into sub-tasks using problem decomposition.

Problem decomposition

We took a 'top-down' approach to tackle our problem and ended up with a list of sub-tasks. These being:

- Detecting a light
- Tracking the light
- Detecting an obstacle
- Avoiding the obstacle.

All these subtasks must be done in parallel. We broke these tasks down even further to better allocate them amongst our group members.

For all the tasks involving sensors, all circuit components and sensors needed to be attached to the robot in a configuration that would allow let our robot be able to achieve its tasks.

Furthermore, learning from our previous project, the code should be written in a way that is easy to understand, easy to test, and implement new features, ideally being able to test each function individually.

Task Allocation

Our final tasks were allocated and the extent of each are as follows:

- Light detection and tracking implementation – Vlad
    - Implementation of the light finding behaviour that occurs at the start of the run.
    - Tracking and following of the light source throughout the rest of the run.
    - Data collection, modelling and calibration of the phototransistors.
- Sensor calibration and object avoidance – Elise
    - Calibration, filtering and processing of all sensors involving obstacle avoidance. These were the 4 IR sensors and ultrasonic sensor.
    - K-map of the obstacle avoid function to allocate robot movement depending on the sensor readings.
- Robot behaviour logic and light extinguishing – Andrew
    - Implementation of the system architecture for robot behavioural control to work. Includes designing the finite state machine our robot would be using and processing all robot movement.
    - Implementing the logic of the light extinguishing function.

- Hardware and mount design – Christoph
    - Placement of all sensors and actuators. These are all the IRs, phototransistors, ultrasonic and fan as well as all the circuit components.
    - Designing and printing all mounts to house these components to keep our robot invulnerable to vibrations and structural issues.

Agile Product Development

Because of our new system architecture, we were successfully able to implement an agile development strategy where we could individually develop and test functions in parallel to each other. This made it easier when merging our individual systems as we know each system works in a vacuum.

For each of our individual tasks, we conducted a sprint involving design and implementation, then finally testing and adjustments to meet our desired requirements.

Some of our main sprints include:

Obstacle avoidance

- We started with designing the logic map and progressed to implementation where we tested the obstacle avoidance system independent of other robot functions.
- Through testing, we iteratively adjusted our system. For example, moving from strafing to turning around objects as to not get stuck in a corner.

Light detection and tracking

- Started with just detecting and tracking a light using the phototransistors attached to the servo.
- Moved onto implementing the ability to move toward and follow the light using the entire body of the robot.

External Software Management

Throughout this project we utilised GitHub and Discord to keep track of the code, tasks, and what steps were still required. We found this approach worked well, with the fundamental code and hardware being completed a week prior to the demonstration which then provided us time for fine tuning the robot.

We used GitHub to host our code, it provided us with the ability to share code easily between multiple devices and allowed us to roll back the code if necessary. For communication we used Discord, which provided an easy solution to message one another any information, data, or images. We also used Discord to summarise what we had done and our next steps during lab sessions to keep us on schedule and track all meeting minutes and progress made & issues encountered during lab sessions.

Even though GitHub, Discord, and Google Docs gave us the necessary tools to keep this project on track, we did face certain challenges. In future utilisation of software Jira, Trello, and/or notion could improve the tracking of tasks, and discourse regarding developments. We found that task status was not able to be ascertained without messaging the whole group which led to minor delays in communication and would lead to some minor ambiguity regarding the task allocation, which at times did lead to unnecessary redundancy.